

SERVERLESS-EDGE TELEMETRY PIPELINE

IoT based Telemetry System

Presented by:

Samanawaya Datta

Department of CSE, Jadavpur University

Session: 2025-2027

CONTENTS

1. Introduction
2. System Architecture
3. Implementation
4. Dashboard Interface
5. Results & Conclusion
6. References

INTRODUCTION

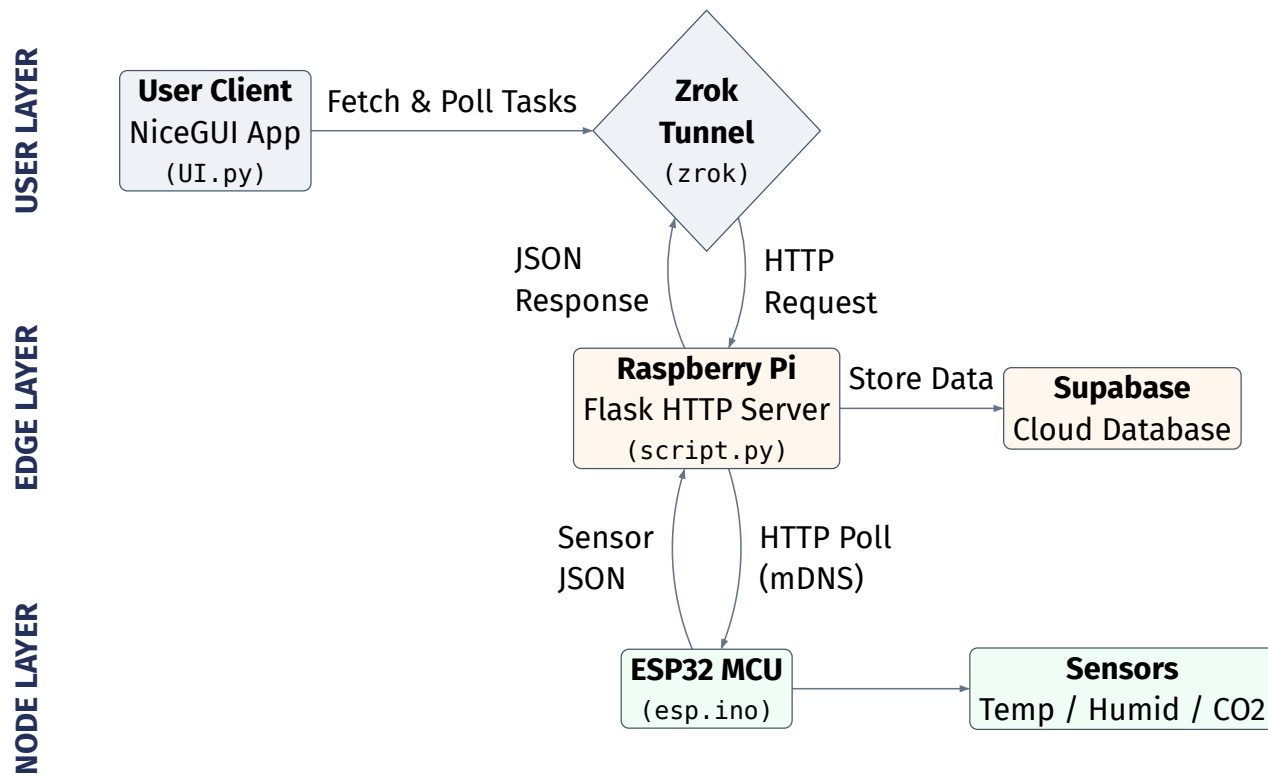
“On-demand fetching, logging, and storage of sensor data, based on serverless tunneling system.”

- Collect environmental data using ESP32 and store it in the cloud database.
- Expose the local API securely via a Zrok tunnel.
- Visualize sensor data using a NiceGUI dashboard.

Goal: Securely expose real-time data without static IP or port forwarding.

- **Serverless-Edge Pipeline:** Designed a lightweight, pull-based architecture that aggregates and processes telemetry data at the edge prior to cloud insertion.
- **Zero-Config Secure Access:** Integrated Zrok tunneling to bypass NAT constraints, provisioning a secure, persistent public endpoint without static IPs.
- **Robust Data Aggregation:** Implemented background time-series collection and edge-side statistical computation to suppress low-cost sensor noise.
- **Interactive Visualization:** Implemented a NiceGUI dashboard featuring live metric tracking, historical logging, and dynamic trend plotting.

SYSTEM ARCHITECTURE



Request-Response Flow:

1. Client Request:

Dashboard requests data via the public Zrok URL.

2. Tunnel Routing:

Zrok securely forwards the HTTP request to the Raspberry Pi's Flask API.

3. Local Polling:

Raspberry Pi queries the ESP32 over the local network using mDNS.

4. Sensor Read:

ESP32 reads physical sensors and returns raw JSON data to the Pi.

5. Edge Processing & Return:

Pi logs readings to Supabase, computes Min/Max/Avg statistics, and returns the aggregated payload to the dashboard.

Figure 1: System architecture

- **Node:** ESP32 (DHT11, MQ-135) + Arduino C++
- **Edge:** Raspberry Pi 4 + Flask API
- **Cloud:** Supabase (PostgreSQL)
- **UI:** NiceGUI Dashboard
- **Tunnel:** Zrok (Zero-trust network access)

IMPLEMENTATION

Workflow & Sampling

1. ESP32 reads sensors
2. Edge gateway polls ESP32
3. Samples collected over duration
4. Data stored in Supabase (per sample)
5. Edge computes Min/Max/Avg statistics

Edge-Computed Statistics

Minimum: $\min(X) = \min(x_1, \dots, x_n)$

Maximum: $\max(X) = \max(x_1, \dots, x_n)$

Average: $\text{avg}(X) = \frac{1}{n} \sum_{i=1}^n x_i$

- Offloads processing from the client UI
- Reduces impact of low-cost sensor fluctuations
- Provides stable CO2 and climate trends

temphum table

Field	Type
id	UUID
temp	FLOAT
hum	FLOAT
status	TEXT
completed_at	TIMESTAMP

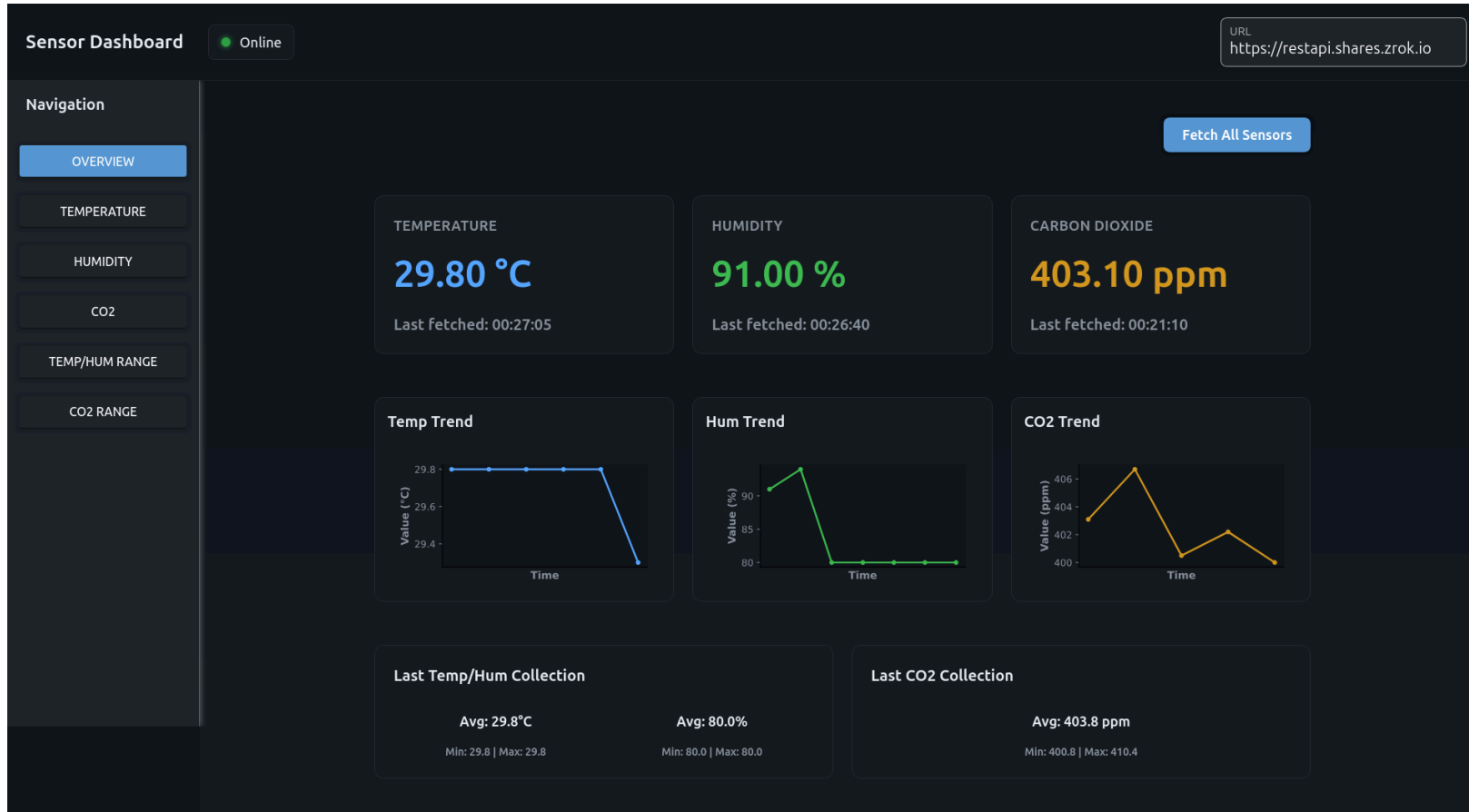
gasval table (CO2)

Field	Type
id	UUID
gas	FLOAT
status	TEXT
completed_at	TIMESTAMP

Service	Purpose	Behavior
esp32-bridge.service	Runs Flask API	Starts after network
zrok-tunnel.service	Creates public endpoint	Auto restart enabled

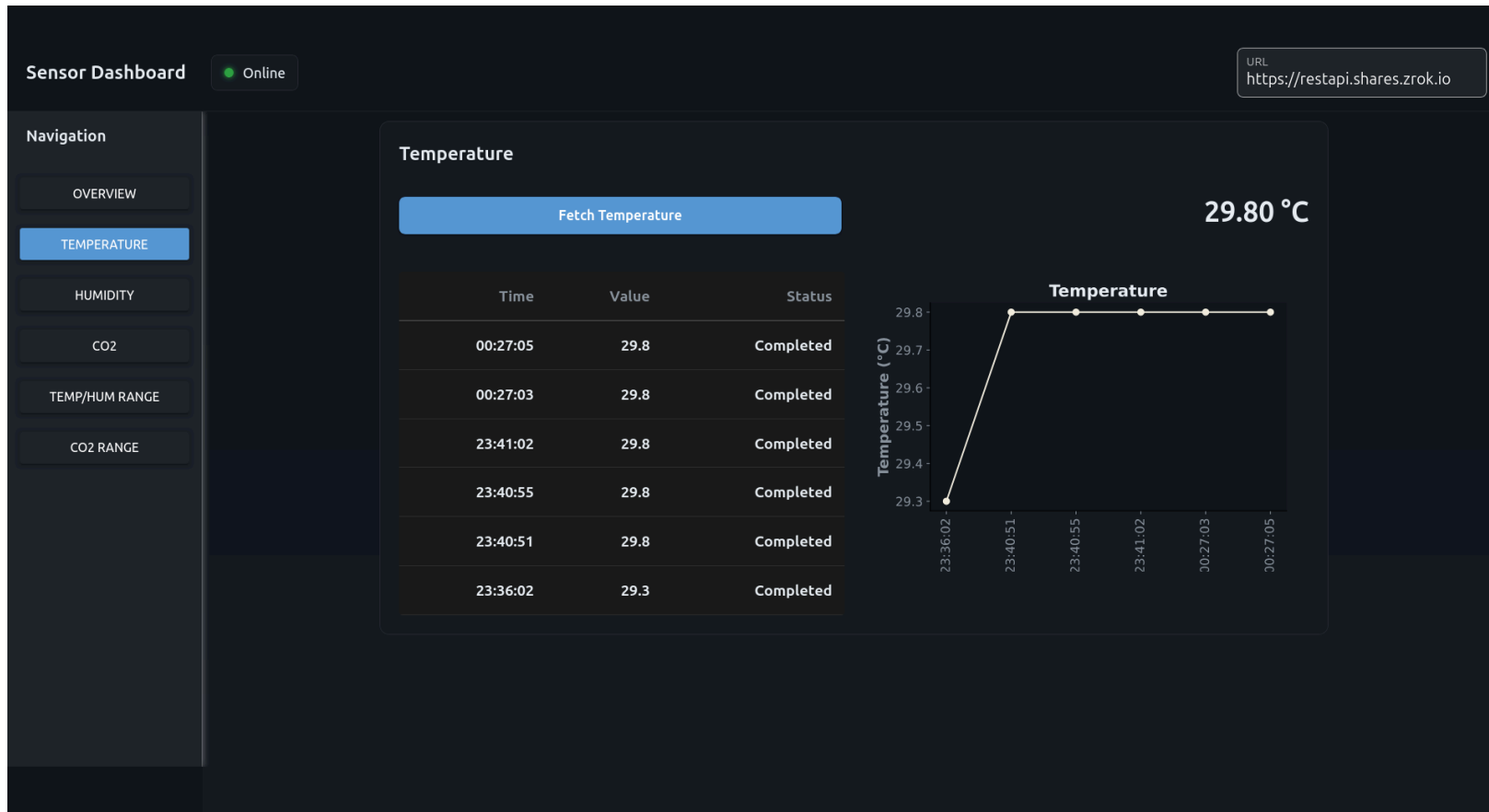
- Fully automatic startup after reboot.
- Failure recovery using restart policies.
- Zero manual intervention required.

DASHBOARD INTERFACE



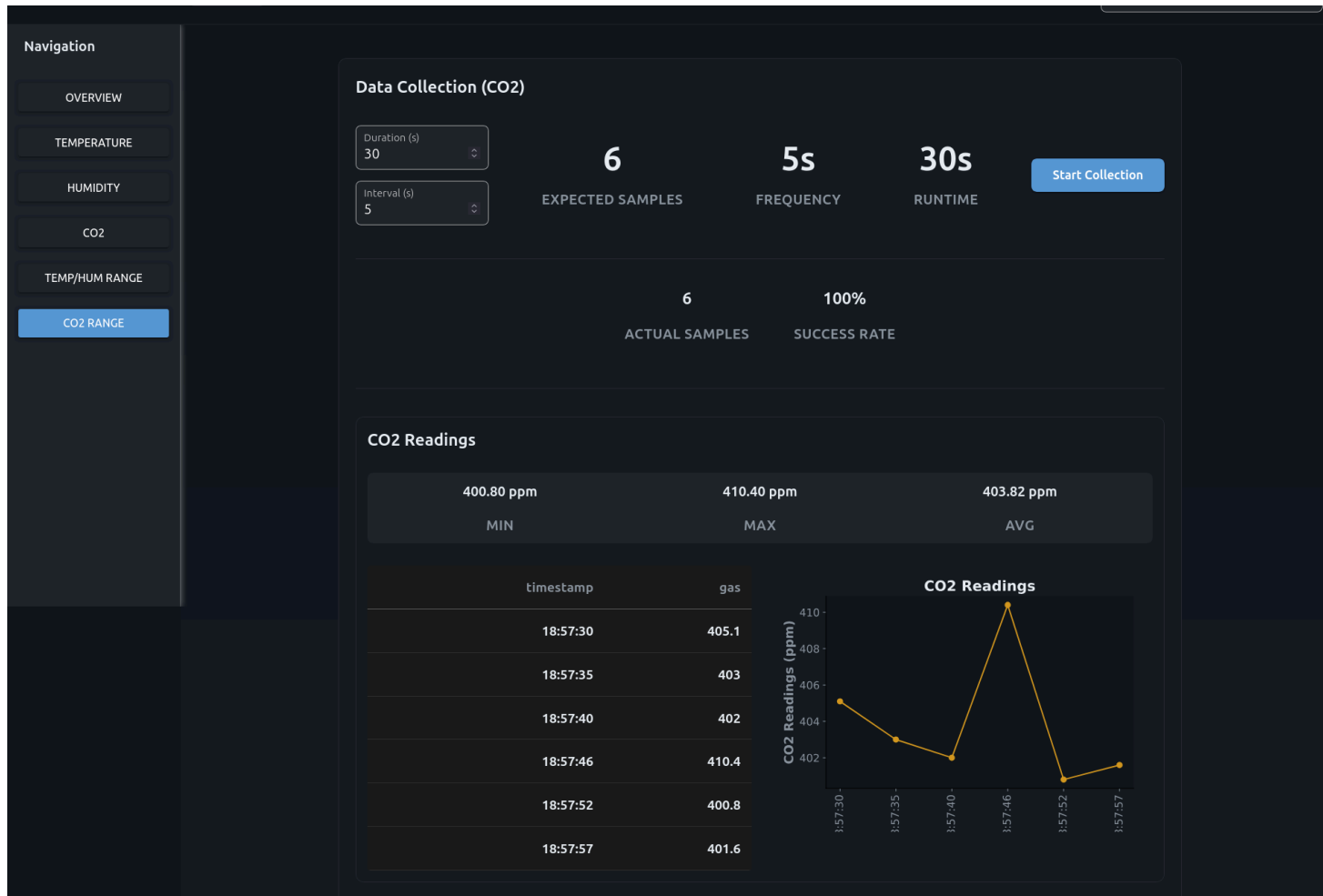
Key Features:

- Unified real-time metric cards
- Master “Fetch All” button
- Mini trends & live status indicator



Key Features:

- On-demand atomic reads
- Interactive trend graphs
- Execution logs & error flags



Key Features:

- Configurable sampling loops
- Non-blocking background tasks
- Edge-computed Min/Max/Avg stats

RESULTS & CONCLUSION

Temp (°C)	Hum (%)	CO2 (ppm)	Status
32.1	79.3	410.6	Completed
33.0	79.9	410.8	Completed
32.8	80.1	409.9	Completed

- Stable sensor readings observed after edge processing.
- Successful and consistent cloud database insertion.
- Reliable remote accessibility via the Zrok tunnel.

Conclusion

- Implemented a secure serverless-edge pipeline with public access via Zrok, eliminating static IPs and port forwarding.
- Maintained stable operation with automated `systemd` recovery and zero-downtime tunnel recreation.
- Achieved comprehensive visualization and cloud integration despite the precision limits of low-cost DHT-11 and MQ-135 sensors.

Future Scope

- Replace DHT11 with BME280 or SHT41/SHT45 for higher precision.
- Add WebSocket support for real-time, push-based updates.
- Implement ML-based anomaly detection for sensor data.
- Enable multi-node deployment and Docker containerization.

REFERENCES

- [1] I. D. N. Araya, M. Gerndt, and S. Benedict, “Serverless IoT Framework,” in *2025 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*, IEEE, Jun. 2025, pp. 130–139. doi: [10.1109/IPDPSW66978.2025.00029](https://doi.org/10.1109/IPDPSW66978.2025.00029).
- [2] S. Bose and N. Mukherjee, “SenslaaS: A Sensor-Cloud Infrastructure with Sensor Virtualization,” in *2016 IEEE 3rd International Conference on Cyber Security and Cloud Computing (CSCloud)*, IEEE, Jun. 2016, pp. 232–239. doi: [10.1109/CSCloud.2016.28](https://doi.org/10.1109/CSCloud.2016.28).
- [3] S. Nastic, P. Raith, A. Furutanpey, T. Pusztai, and S. Dustdar, “A Serverless Computing Fabric for Edge & Cloud,” in *2022 IEEE 4th International Conference on Cognitive Machine Intelligence (CogMI)*, 2022, pp. 1–12. doi: [10.1109/CogMI56440.2022.00011](https://doi.org/10.1109/CogMI56440.2022.00011).
- [4] A. Mujezinović and V. Ljubović, “Serverless architecture for workflow scheduling with unconstrained execution environment,” in *2019 42nd International Convention on Information and Communication Technology, Electronics and Microelectronics (MIPRO)*, 2019, pp. 242–246. doi: [10.23919/MIPRO.2019.8756833](https://doi.org/10.23919/MIPRO.2019.8756833).
- [5] N. S. Kumar and S. S. Selvakumara, “Serverless Computing Platforms Performance and Scalability Implementation Analysis,” in *2022 International Conference on Computer, Power and Communications (ICCP)*, 2022, pp. 598–602. doi: [10.1109/ICCP55978.2022.10072137](https://doi.org/10.1109/ICCP55978.2022.10072137).

- [6] Z. Al-Ali et al., “Making Serverless Computing More Serverless,” in *2018 IEEE 11th International Conference on Cloud Computing (CLOUD)*, 2018, pp. 456–459. doi: [10.1109/CLOUD.2018.00064](https://doi.org/10.1109/CLOUD.2018.00064).
- [7] Pallets, “Flask Documentation.” Accessed: May 23, 2026. [Online]. Available: <https://flask.palletsprojects.com/>
- [8] Supabase, “Supabase Documentation.” Accessed: May 23, 2026. [Online]. Available: <https://supabase.com/docs>
- [9] Zauberzeug, “NiceGUI Documentation: Let developers write web UIs with Python.” Accessed: May 23, 2026. [Online]. Available: <https://nicegui.io/>
- [10] OpenZiti, “zrok Documentation.” Accessed: May 23, 2026. [Online]. Available: <https://docs.zrok.io/>

Thank You